

# GRAN → DiGress: Upgrade Journey

**Background:** We're trying to auto-generate "bubble diagrams" — abstractions of apartment floorplans where each room is a node (one of 7 types: Living / Bedroom / Bathroom / Kitchen / Balcony / Storage / External) and adjacent rooms are connected by an edge. Generated graphs need: - **Structurally plausible:** node count, edge count, degree distribution match real floorplans - **Semantically plausible:** typically one Living room per plan, usually the most-connected hub; small rooms (Bathroom, Balcony) appear at realistic rates

Three main metric families: - **MMD** — does the structural distribution look like real data? (lower = better) - **endpoint pair KL** — does the joint distribution of "room types at the two ends of each edge" look real? (lower = better) - **per-class room frequency** — are Living/Bedroom/Bathroom etc. proportions reasonable?

## 1. TL;DR

**GRAN family (v2 + multiple patches): every variant trades one problem for another.**

Structure is learnable, but the link between "what type of room a node is" and "how it connects to other nodes" is never properly captured: either Living rooms fail to act as hubs, or small rooms collapse out of the distribution.

**DiGress (discrete diffusion): the right direction.** Node types and edge types are denoised by the same network, so structure and semantics co-evolve.

| Experiment               | What changed                         | One-line result                                          |
|--------------------------|--------------------------------------|----------------------------------------------------------|
| gran_v2_rplan (baseline) | Vanilla GRAN v2                      | Living-Living edges way too common (9.85% vs 0.24% real) |
| gran_v2_rplan_struct     | Drop room-type loss, structure only  | Structure OK, but Living almost disappears (3.3%)        |
| gran_v2_rplan_deonly     | Add a node-degree supervision signal | Type distribution drifts, Bathroom nearly gone           |

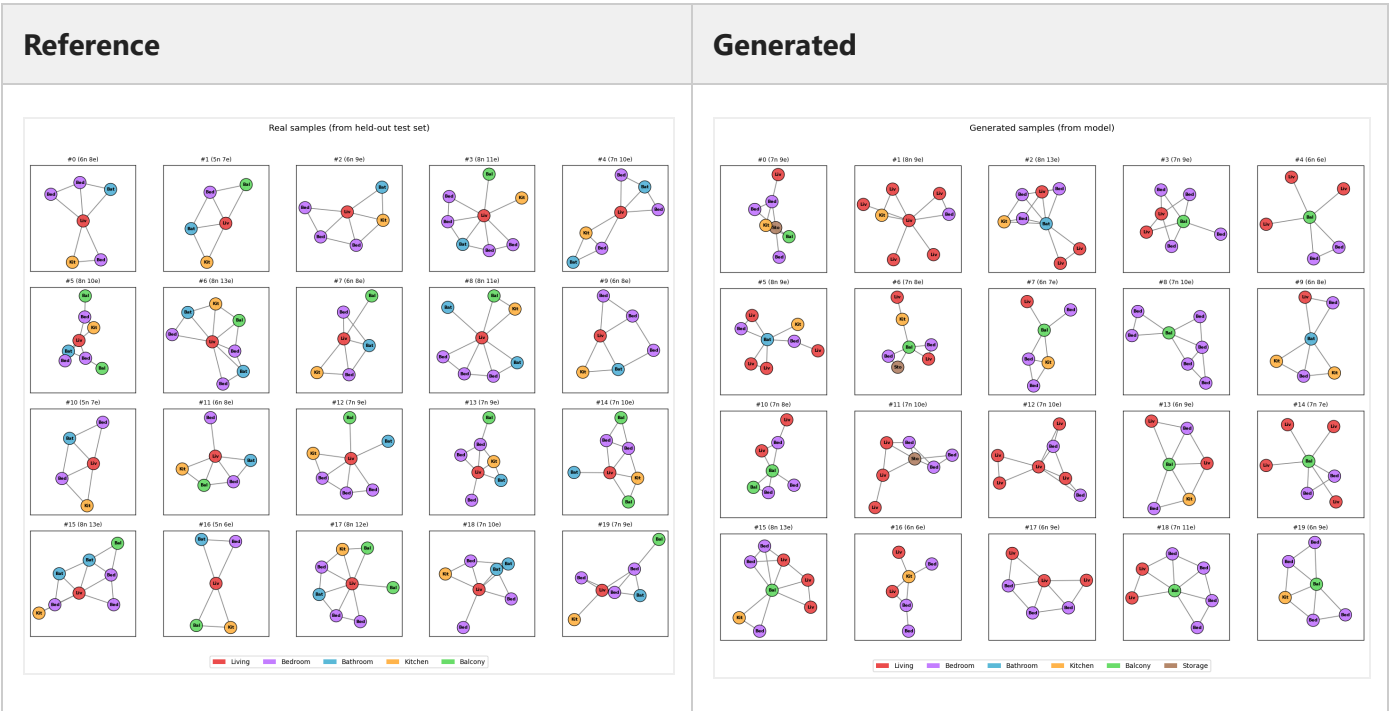
| Experiment                | What changed                                 | One-line result                                              |
|---------------------------|----------------------------------------------|--------------------------------------------------------------|
| gran_v2_rplan_p1          | Reweight type loss                           | Bedroom blows up to 63% (real 36%)                           |
| gran_v2_rplan_attrbalance | Class-weighted type loss                     | Worse — endpoint-pair KL up to 0.81                          |
| gran_v2_rplan_planc       | Edge head conditioned on endpoint room types | Living-Living edges drop sharply, but small classes collapse |
| DiGress                   | Joint denoising of nodes + edges             | Structure + semantics converge together                      |

2. GRAN runs in detail

2.1 Baseline ( gran\_v2\_rplan )

**What we did:** Standard GRAN v2 — generate one node at a time. For each new node, jointly predict (a) its room type and (b) which earlier nodes to connect to.

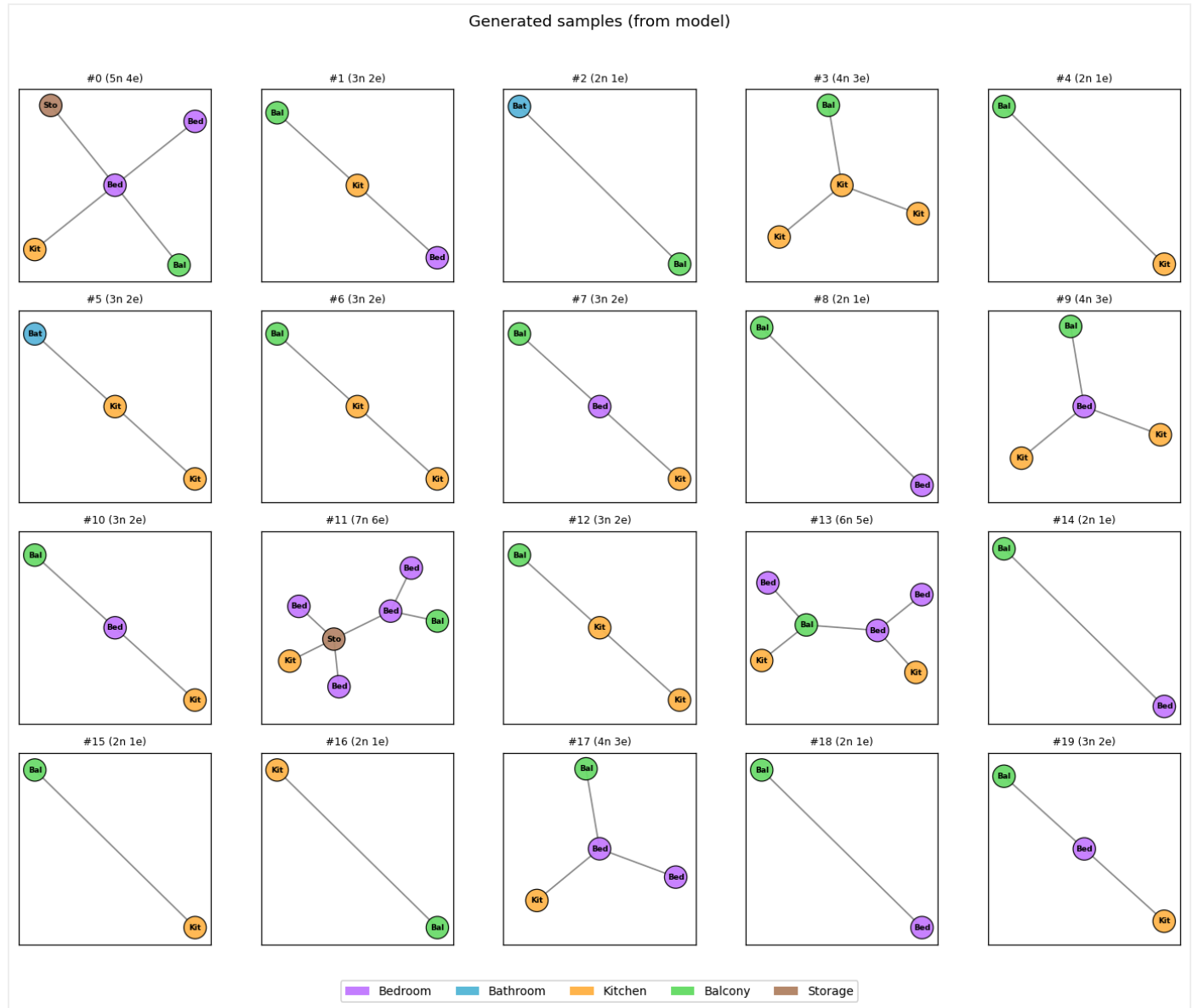
**Result:** - Structural MMD is fine; node/edge count distributions look real - **Problem:** Living-Living edges occur in **9.85%** of all edges (real: 0.24%). The model frequently produces floorplans with two Living rooms next to each other, which is unrealistic.



## 2.2 Struct-only ( gran\_v2\_rplan\_struct )

**What we did:** Drop the room-type loss entirely; learn only the connectivity, to see whether structure can be learned cleanly on its own.

**Result:** Connections look reasonable, but with no type supervision the model defaults most nodes to Bedroom. Living drops to 3.3% — effectively a "draws walls but can't label rooms" model.

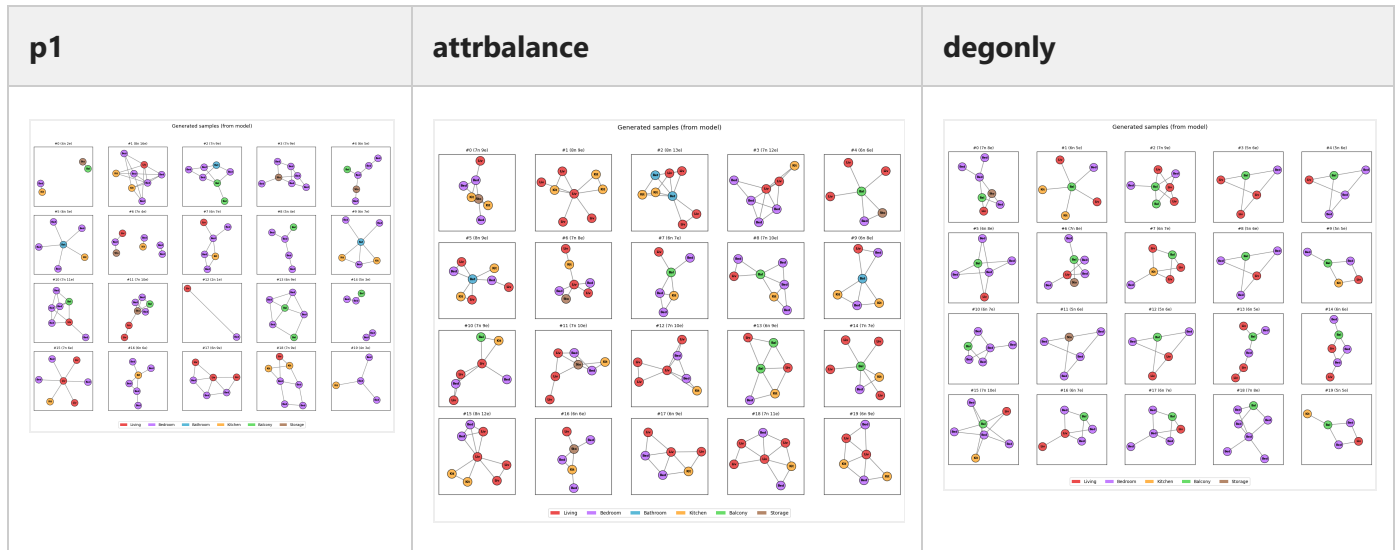


## 2.3 Plan-1 / Plan A / Degree-only

Three independent attempts, each targeting one symptom:

- **p1** (raise the room-type loss weight): the model collapses uncertain nodes into Bedroom → **Bedroom = 63%** (real 36%)

- **attrbalance** (class-weighted type loss to boost rare rooms): backfires — endpoint-pair KL rises to **0.81**
- **degonly** (extra supervision on each node's degree): Balcony gets close to real (19% vs 14.7%), but Bathroom collapses to 0.5%



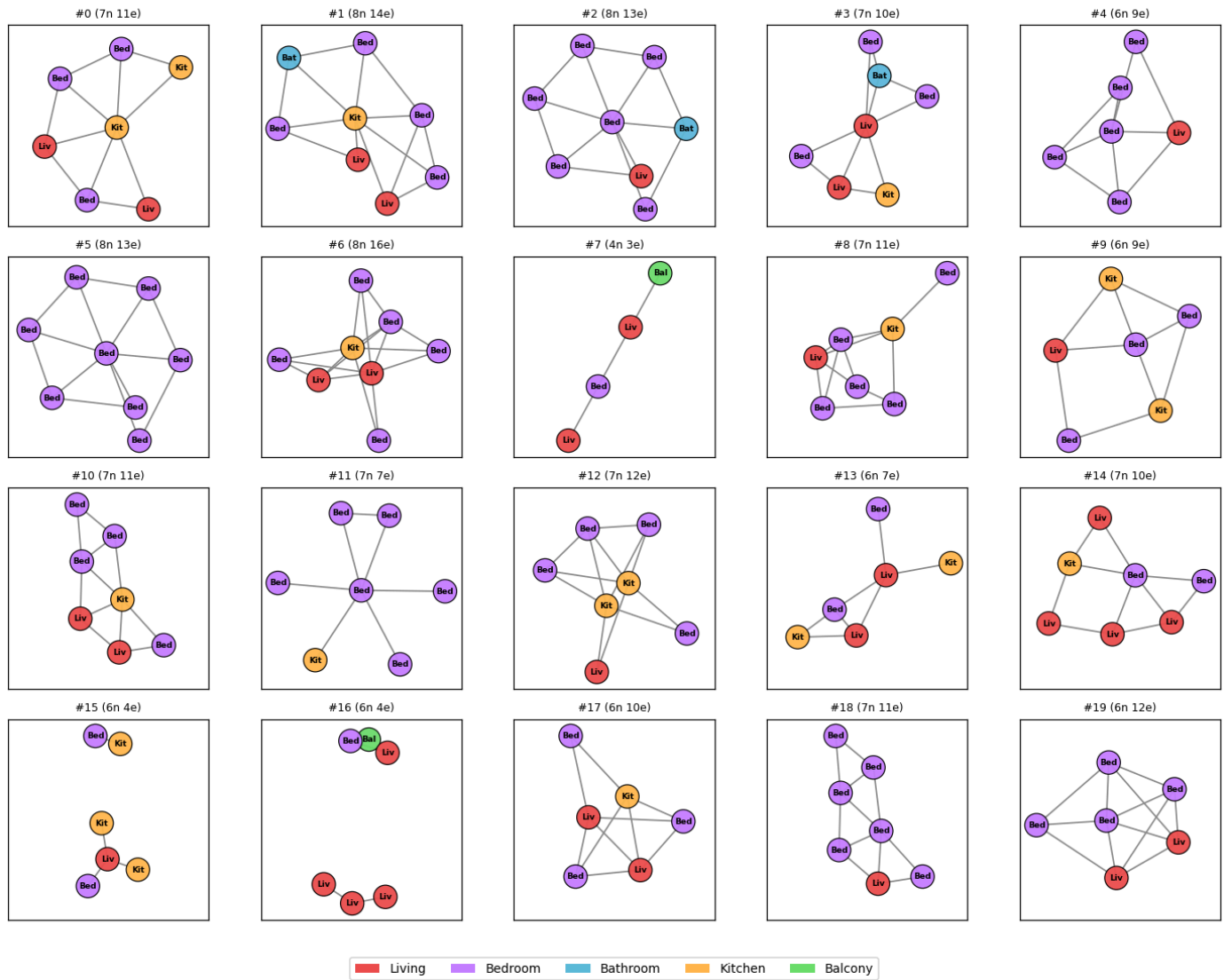
## 2.4 Plan C-1: Edge head sees endpoint room types (the deepest attempt)

**What we did:** Originally, when the model decides whether two nodes should be connected, it only looks at their hidden vectors. Plan C **also feeds in the room-type embedding for both endpoints** — telling the edge predictor "endpoint A is Living, endpoint B is Bedroom — now decide if they connect."

**Partial wins:** - Living-Living edge ratio **9.85% → 4.75%** ✓ (closer to real 0.24%) - KL of "how many Living rooms per graph" distribution **1.40 → 0.53** ✓ (closer to real, where there's basically always exactly one Living)

**New problem:** The edge predictor learns a shortcut — "I'm not confident about Bathroom/Balcony combinations, so just generate fewer of them." Result: **Bathroom = 2.0% (real 18%), Balcony = 1.3% (real 14.7%)** — class collapse.

Generated samples (from model)



### 3. Why GRAN can't get there

GRAN is **autoregressive + two-headed**:

```

current node → GNN → hidden h
                |
                |— edge head → who to connect
                |— type head → what room type
  
```

The two heads **don't see each other's predictions**; training just sums their losses. "Joint" only happens at the **loss level** — at the **model level the two are decoupled**. Plan C-1 wired up the type → edge direction, fixing one half:

- Edge side learned "Livings shouldn't be adjacent" ✓

- Type side never learned "preserve rare rooms" ❌
- The model converged to a "**sacrifice room diversity to satisfy structural constraints**" local optimum

## 4. DiGress is the answer

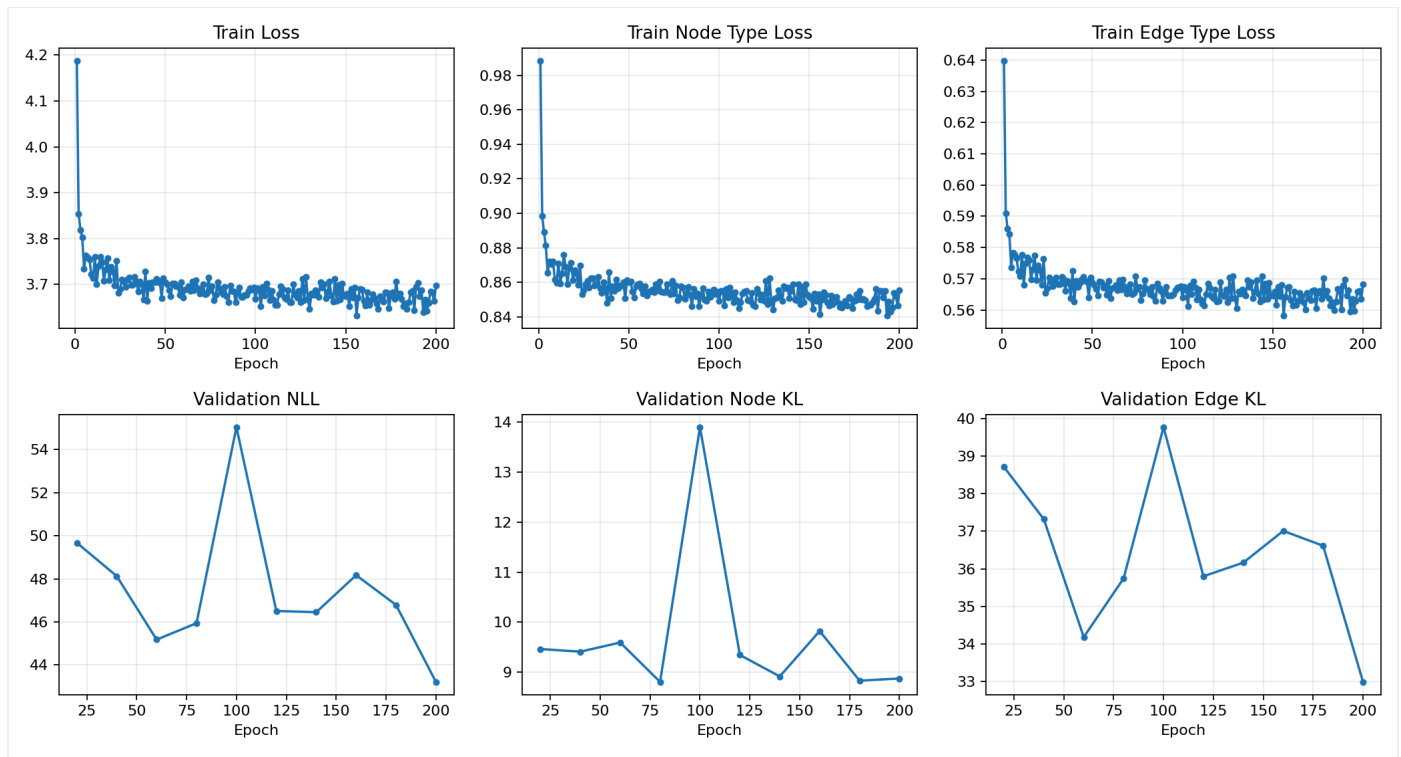
### Architecture in brief:

- **Generative model:** discrete diffusion (D3PM). Forward process gradually corrupts node-type and edge-type categories over  $T = 500$  steps using **marginal** transitions (cosine schedule). Reverse process: a network takes noisy  $(X_t, E_t)$  and predicts clean categories  $(X_0, E_0)$ ; sampling iterates this in parallel over all nodes and edges.
- **Backbone:** 5-layer **Graph Transformer**. Each layer maintains three feature streams — node  $X$ , edge  $E$ , global  $y$  — and updates them jointly: node attention is gated by edge features, edge updates are modulated by node interactions,  $y$  aggregates from both. This is the key difference from GRAN: **nodes and edges condition each other inside every layer**, not just at the loss.
- **Extra features** ( `extra_features: all` ): cycle counts (3–6) per node + Laplacian spectral features are concatenated into the inputs each step, so the network doesn't have to relearn them from scratch.
- **Training on RPLAN:** 200 epochs, batch 128, AdamW lr  $2e-4$ , hidden dims `dx=256, de=64`, 8 heads.

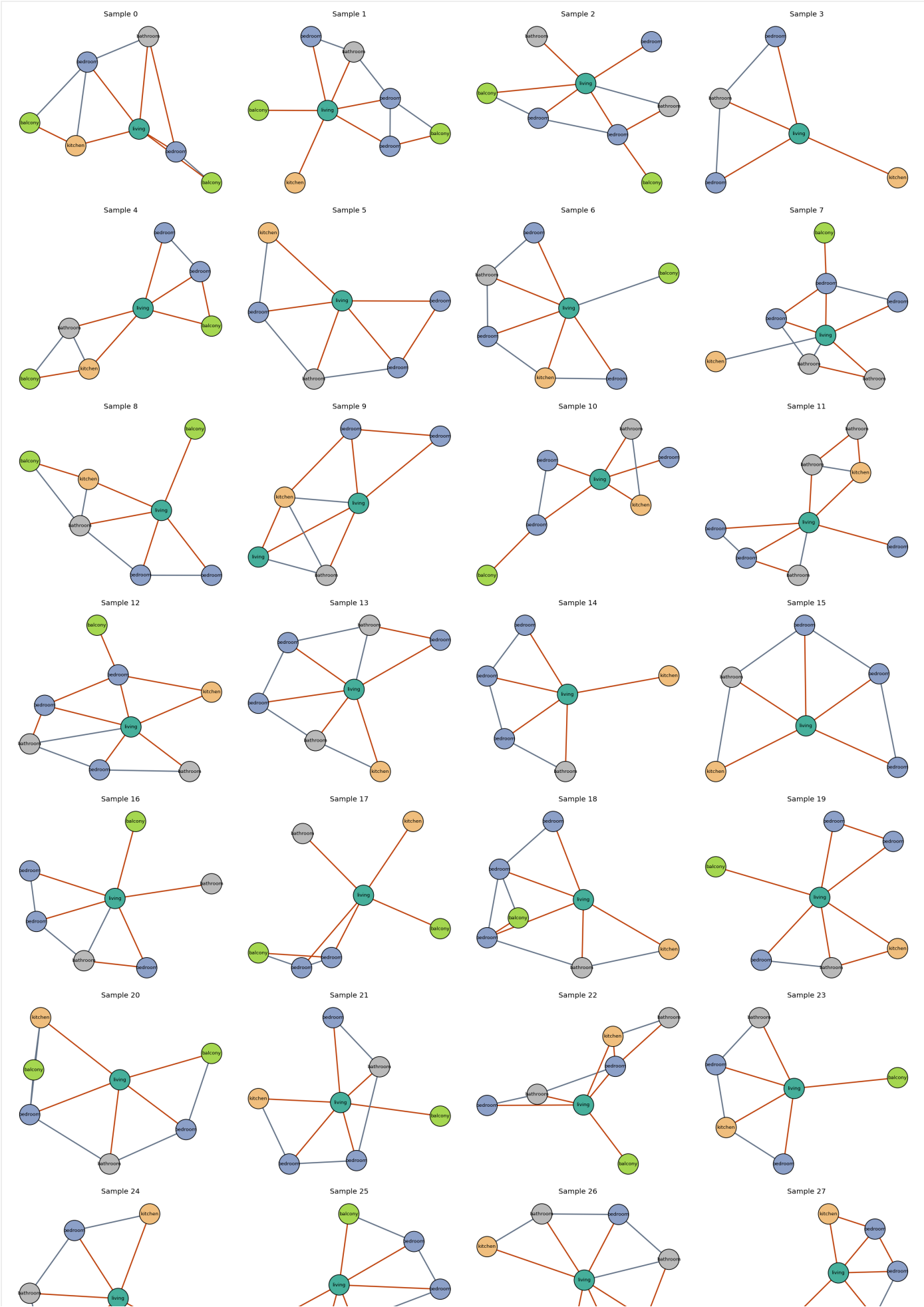
**Why this beats GRAN:** node-type prediction sees the current edge state, and edge prediction sees the current node types — true bidirectional coupling at every step, not just two heads sharing a loss.

Current DiGress run on RPLAN ( `outputs/2026-04-23/21-11-23-rplan` ):

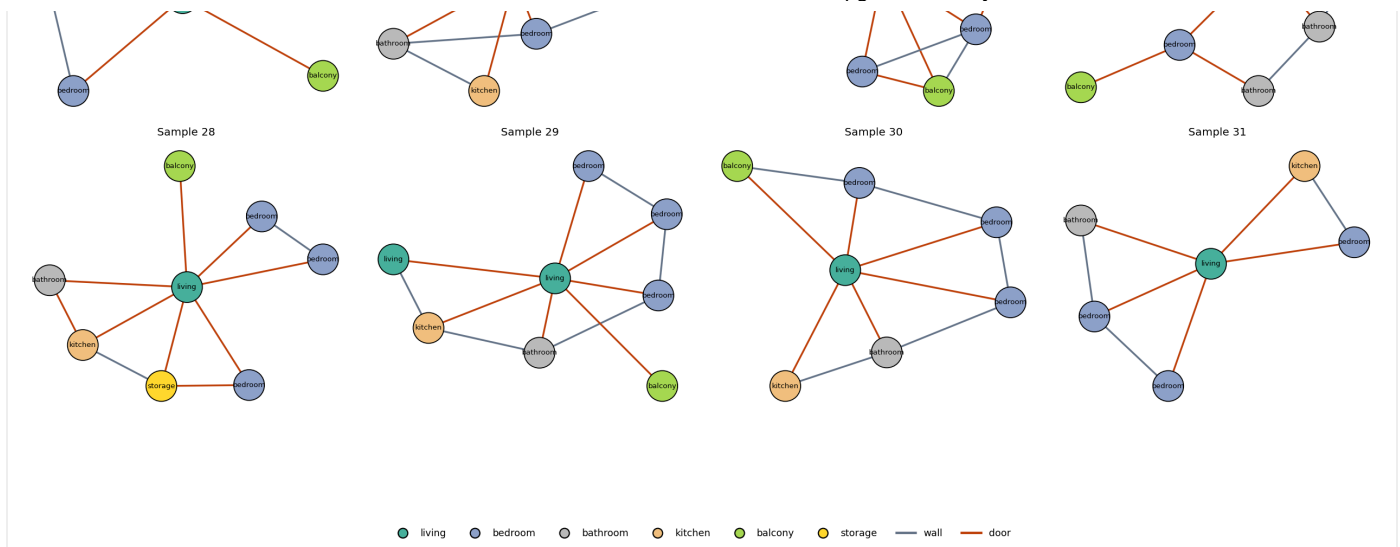
Training curves (converged):



Sample grid at epoch 199 (structure + semantics both closer to real than any GRAN variant):







## 5. Next steps

Pivot to **MSD (Modified Swiss Dwellings, ECCV 2024)** — multi-apartment building complexes, much larger and more diverse graphs than RPLAN — with an upgraded DiGress. Detailed plan in [docs/absorbing\\_d3pm\\_msd\\_plan.md](#). Three core upgrades:

- **Absorbing D3PM:** change the noising process from "marginal" to "absorbing." In current DiGress, the forward process replaces a node/edge category with another category drawn from the dataset marginal. In **absorbing** D3PM, the forward process instead replaces it with a special `[MASK]` token — once a position becomes `[MASK]`, it stays `[MASK]` until the reverse process unmask it. This is mathematically equivalent to BERT-style masked modeling on graphs, which means one trained model handles three tasks just by varying which positions are pre-masked at inference: (A) all masked → unconditional generation; (B) some nodes/edges given, rest masked → partial-graph completion; (C) almost everything given, one node masked → attribute prediction. No separate heads, no post-hoc hacks.

*Why not just do partial completion with vanilla (marginal) DiGress?* It is possible via **RePaint** (Lugmayr et al., CVPR 2022) — a technique originally for image inpainting. At each reverse step, the known region is re-noised to the current timestep and pasted over the model's prediction, so only the unknown region actually uses the model's output; a periodic "jump-back + resample" harmonizes the boundary. DiGress Appendix D.1 adapts this for graphs. The problem: the model was trained only on "all-noise → clean," so at inference time we're feeding it a "partially-known + partially-noise" input that's out-of-distribution, and quality depends on tuning jump-length / resample-count. Absorbing D3PM sidesteps this entirely — the training objective at each timestep already **is** "given some known positions and some `[MASK]` positions, predict the `[MASK]` ones," so partial completion is a native training-distribution task, not an inference-time workaround. (The training set itself doesn't change;

the forward noising process, by masking each position independently with probability  $\beta_t$ , naturally produces inputs with every mask ratio from 0% to 100% across training steps — same trick as BERT's random-15%-mask, just swept over all ratios.)

- **Graphormer centrality encoding:** a node-feature add-on borrowed from the Graphormer paper (NeurIPS 2021). For each node, compute (i) its **degree** (number of edges it has) and (ii) its **eigenvector centrality** (a graph-theoretic "importance" score, high for nodes connected to other important nodes). Both are turned into learnable embeddings and concatenated to the node features before the transformer. This bakes in the "Living room is the hub" prior — the network is told upfront which nodes are central.
- **MaskGIT-style parallel decoding:** a faster sampler from MaskGIT (CVPR 2022). Standard D3PM does 500 sequential reverse steps, unmasking everything every step. MaskGIT instead runs ~10–20 steps; each step (i) predicts categories at all currently-masked positions, (ii) keeps only the **top-k most confident** predictions and commits them, (iii) leaves the less-confident positions masked for the next step. Result: ~30× fewer steps with comparable or better quality, because the easy decisions get locked in early and the network can focus capacity on the hard ones.